Department of Computer Science & Engineering
The University of Texas at Arlington

Detailed Design Specification
CSE 4317: Senior Design II
Spring 2026

ProtoBot
StarCraft Brood War Game Playing Agent

David Daffern
Marshall Helton
Aadhitya Kumar
Preston Loera
Matthew Nguyen
Tung Nguyen

Revision History

Revision	Date	Author(s)	Description
0.1	2.01.2026	PL	document creation
0.2	2.13.2026	PL AK	first draft
1.0	2.13.2026	PL	complete draft
1.1	3.29.2026	PL	revised some sections
1.2	4.11.2026	PL, MH, MN	further revisions to document
2.0	4.11.2026	PL, MN	updated document

Contents

1 Introduction **5**

2 System Overview **5**

3 Commander **6**
3.1 Layer Hardware 6
3.2 Layer Operating System 6
3.3 Layer Software Dependencies 7
3.4 Subsystem: Timer Manager 7
3.5 Subsystem: A* Path Generator 8
3.6 Subsystem: Resource Requests 8

4 Information Manager **10**
4.1 Layer Software Dependencies 10
4.2 Subsystem: Influence Map 11

5 Scouting Manager **12**
5.1 Layer Software Dependencies 12
5.2 Subsystem: Probe Scout 12
5.3 Subsystem: Zealot Scout 13
5.4 Subsystem: Observer Scout 14
5.5 Subsystem: Dark Templar Scout 15

6 Economy Manager **16**
6.1 Layer Software Dependencies 16
6.2 Subsystem: Nexus Economy 16

7 Building Manager **18**
7.1 Layer Software Dependencies 18
7.2 Subsystem: Building Placer 18
7.3 Subsystem: Builder 19
7.4 Subsystem: Buildings 19

8 Combat Manager **20**
8.1 Layer Software Dependencies 20
8.2 Subsystem: Squads 20

9 Strategy Manager **22**
9.1 Layer Software Dependencies 22
9.2 Subsystem: Spender Manager 23
9.3 Subsystem Data Processing: Unit, Upgrade, and Building Production 23

10 Appendix **25**

List of Figures

1 ProtoBot architecture 6
2 Commander subsystem diagram 7
3 Information Manager subsystem diagram 10
4 Scouting Manager subsystem diagram 12
5 Economy Manager subsystem diagram 16
6 Building Manager subsystem diagram 18
7 Combat Manager subsystem diagram 20
8 Strategy Manager subsystem diagram 22
9 Example of tiles in StarCraft 25
10 Example of Fog of War in StarCraft 26
11 Example of sight range in StarCraft 27
12 Protoss unit tech tree 28
13 Protoss building tech tree 29
14 StarCraft resources (minerals and vespene gas) 30
15 Example of buildings being "remembered" by StarCraft map 31

1 Introduction

ProtoBot is an autonomous *StarCraft: Brood War* game-playing agent designed to demonstrate intelligent decision-making in StarCraft's real time environment. Using the Brood War API (BWAPI) to interface with the game, ProtoBot utilizes a variety of artificial intelligence algorithms to handle tasks such as path-finding, resource management, and unit manipulation. The goal of this DDS is to refine the information present in the Architectural Design Specification (ADS) and provide high-level implementation details from the System Requirements Specification (SRS) down to modules, algorithms, and events, detailing how information flows from a StarCraft BWAPI event through ProtoBot's modules.

2 System Overview

The way information is passed from StarCraft to ProtoBot is the previously mentioned BWAPI events. This allows each frame of StarCraft to wait for a "response" from both ProtoBot and the opponent bot before proceeding to the next frame. Eleven BWAPI events are passed to each bot:

- `MatchStart`
- `MatchFrame`
- `MatchEnd`
- `UnitShow`
- `UnitHide`
- `UnitCreate`
- `UnitMorph`
- `UnitDestroy`
- `UnitRenegade`
- `UnitComplete`
- `SendText`

ProtoBot receives these events through the commander and acts as the main way BWAPI events are passed to ProtoBot's submodules. Not all BWAPI events are further sent to ProtoBot's modules, but for the sake of simplicity of this document, the reader can assume that all events are sent to each module, but some events may not be used by ProtoBot's modules.

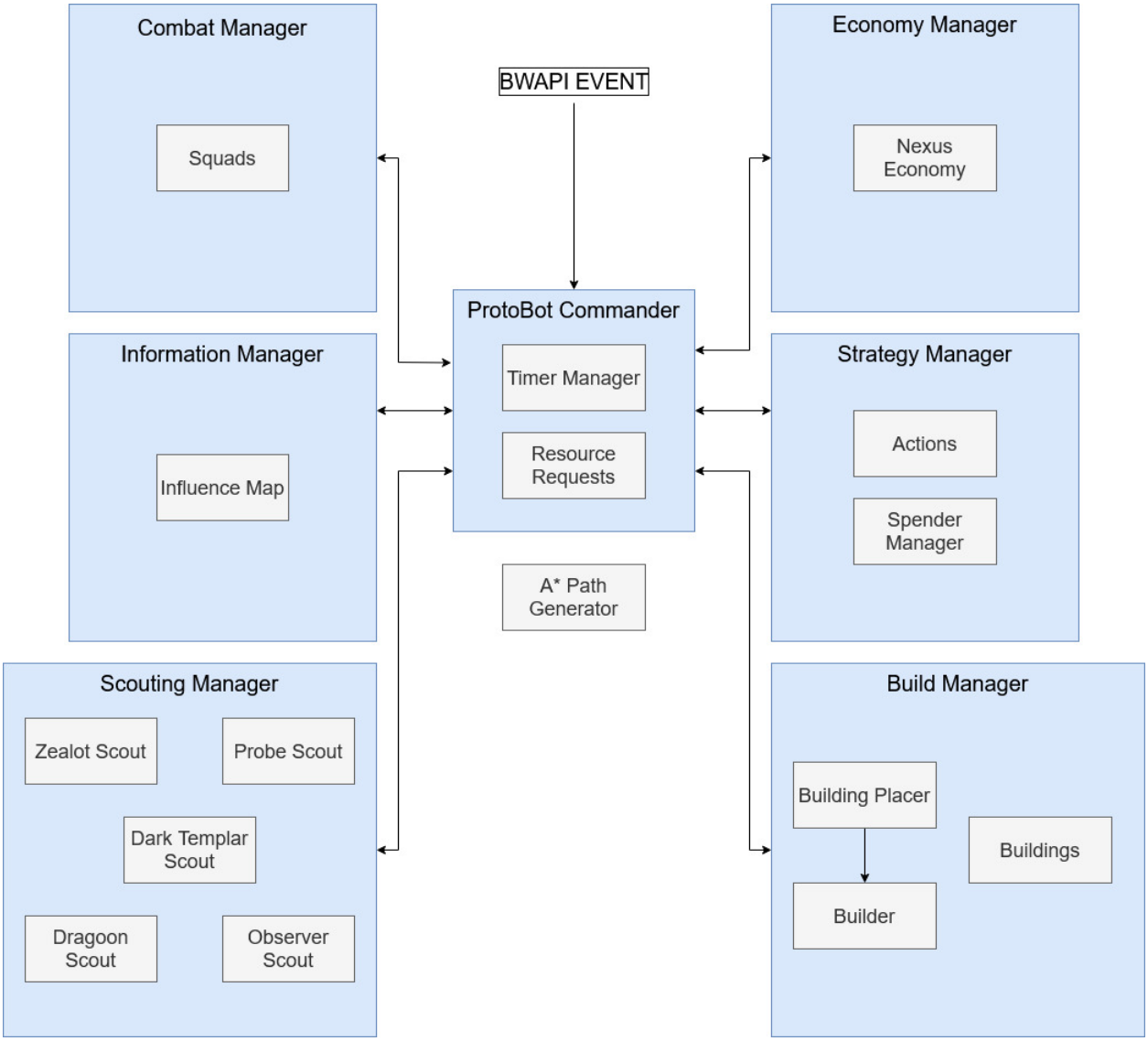

Figure 1: ProtoBot architecture

3 Commander

The commander acts as the way all information from BWAPI is passed to ProtoBot's modules and has no game intelligence besides passing BWAPI events to its sub components in an order that allows ProtoBot's modules to execute intelligent instructions.

3.1 Layer Hardware

A device capable of running StarCraft: BroodWar and BWAPI is required.

3.2 Layer Operating System

BWAPI requires a device has Windows or Linux as its operating system to run.

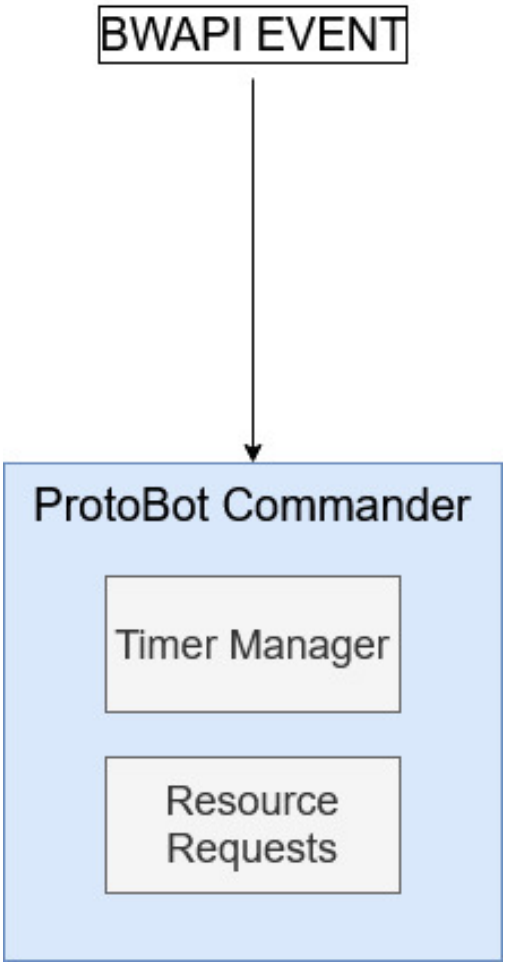

Figure 2: Commander subsystem diagram

3.3 Layer Software Dependencies

Visual Studio 2022: Used to edit, compile, and create an executable of our bot.
StarCraft: BroodWar 1.16.1 executable: The game which the bot plays.
StarCraft data: Information such as maps, BWAPI data, and replays collected during and after local games.
Brood War API (BWAPI): API used to interface our code with StarCraft.
BWAPI.dll: Injection that is made to allow StarCraft to be interfaced with code.

3.4 Subsystem: Timer Manager

The commander does however have a subsystem called the TimerManager and is used to make sure our bot does not take over 42ms per frame. We designed our bot with the AIIDE's rule set in mind and is a useful tool to address long computational times of ProtoBot's modules. Other than for debugging, the TimerManager is not utilized for any decision making in the game. Any actions we want to happen at a specific "time" will be based on StarCraft's frames per second.

3.5 Subsystem: A* Path Generator

For the sake of simplicity of the diagram, we will define the "Astar PathFinder" here in the commander. This will prevent having to define A* multiple times in this document and provides a singular reference on how paths are being generated in ProtoBot's modules that utilize units.

Due to *StarCraft: Brood War* being released in 1998, the game has many issues that require specific algorithms tailored to StarCraft to improve the performance of tasks like unit management.

To address this issue, a static class named AStar has been defined so that any module can use its functionality. The algorithm is a modified version of A* used in graph theory and uses some modifications to make computation times under the required times and path manipulation more ideal. Since not all managers command units, A* is not nessecary for to be defined for them.

To specify how nodes are created for A*, we must first define some StarCraft measurements necessary for nodes [Figure 9]. When a map is created, 32 * 32 pixel tiles are used to populate the map's grid with tiles. StarCraft 1v1 maps can range from 64 * 64 tiles to 256 * 256 tiles. Tiles have many other properties, but for the sake of this problem we are primarily concerned about if a tile is walkable or unwalkable. A tile being walkable means that a unit can move to that tile. Whereas tiles may be non-walkable for a variety of reasons: a building may occupy the tile, terrain elevation may prevent traversal, or the type of tile may be unwalkable (water, lava, etc.). To generate a path, we take in the position of a unit which is the 1 * 1 pixel value of the unit's location on the map, the ending position they are wanting to arrive at, and the type of unit they are. Different units have different collision sizes, so determining whether a tile is walkable or not depends on the size of a unit. From this information, a path of tile positions are then generated using normal A* calculations and returned to the unit that requested the generated path.

3.6 Subsystem: Resource Requests

Resource requests are the way ProtoBot's strategy module decides how we should spend our resources. There are four types of request that can be made: Unit requests, Building requests, Upgrade requests, and Tech requests. With each request potentially requiring different prerequisites to have already be completed before we can create more advanced units. Each request made, depending on its type, transitions through at least three of the four states:

- `PendingApproval`
- `Approved_InProgress`
- `Approved_BeingBuilt`
- `Accepted_Completed`

Building requests are the only request type that utilizes all four states. This is due to Build requests being the only request type that happens asynchronously. Unit, Tech, and Upgrade requests, when approved, happen within the next couple of cycles of StarCraft's game engine. Building requests, on the other hand, require a valid placement to be identified on the StarCraft map, as well as an available worker to travel to that tile and construct the building. All requests issued by ProtoBot's Strategy module are handled by the Building module, which is responsible for determining suitable placements and assigning an available worker for construction. Once a worker is assigned, the request transitions to the state Approved_BeingBuilt until the building is placed or the worker is destroyed. Other requests

are validated again to ensure they remain possible before transitioning to the next state Accepted_-Completed. Further explanation of Resource Requests will be given in the Strategy Manager and Build Manager sections of the DDS.

3.6.1 Subsystem Software Dependencies

BWAPI
BroodWar Easy Map (BWEM)
BroodWar Easy Builder (BWEB)
Other C++ libraries are defined.

3.6.2 Subsystem Programming Languages

C++ is the language that is used to control ProtoBot's actions in StarCraft
ProtoBots code structure utilized Dr.Churchill's STARTcraft as a baseline to add our own work from the provided Visual Studio project.

4 Information Manager

When a standard game of StarCraft is played, a game design element called the "Fog of War" [Figure 10] is used to prevent any information that a player would gain by moving their screen around the map. The only way players can see in this fog is by moving units around the map or placing buildings. Every unit has what is called a "sight range" [Figure 11], a radius around the center of a unit where a unit can see before the fog of war obscures any further vision. To make StarCraft easier for humans, the map can "remember" what a player has seen by drawing the information of buildings found on the map when they are later obscured by the "Fog of War" again [Figure 15]. When the player gains vision over the location, the map will automatically update itself to reflect the latest state of the revealed areas before the player loses vision. That said, ProtoBot is not able to utilize the information from the map, making it important to track anything revealed on the map and then use this information to be able to know where to attack and what strategy the opponent is going for. This is the primary concern of the information manager and is solely responsible for keeping track of all the information we learn about the current state of a game. Information Manager uses data structures to keep track of the opponent's base locations, unit count, upgrades, and tech. The same is true for our units as well; in order to reduce computational times of looping over every unit we can currently see, data structures to keep track of our own units are utilized for ease of access.

4.1 Layer Software Dependencies

BWAPI
BWEM
Misc. C++ Libraries

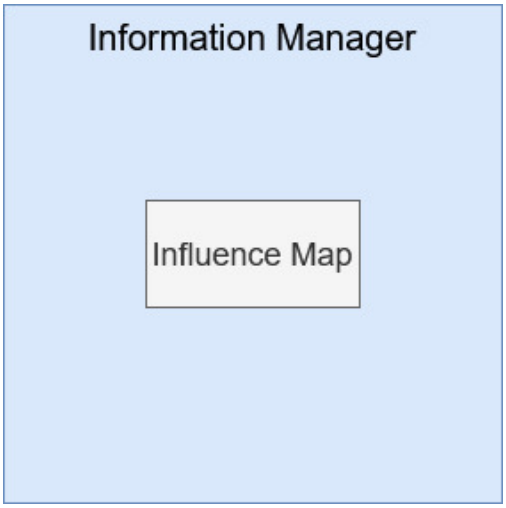

Figure 3: Information Manager subsystem diagram

4.2 Subsystem: Influence Map

The influence map is used to be able to consider the tiles the opponent has control over. Utilizing sight range, we factor in how much influence they have over a tile based on the type of unit they are and their damage if we were to fight them. This is used to be able to find openings in the base for flanks and attacks on a opponents workers and for squads to be able to flank around an army.

4.2.1 Subsystem Software Dependencies

BWAPI
BWEM
Misc. C++ Libraries

4.2.2 Subsystem Programming Languages

C++

4.2.3 Subsystem Data Structures

A data structure for unit counts of our own bots and enemy units is updated over the course of the game. The information we learn is stored in a struct and is just a simple counter or a boolean value on units, upgrades, or tech the opponent has. The more interesting data structures come from tracking the building and unit information of the enemy. Requiring a unique identifier to prevent double-counting, and can return the last known position of the unit.

4.2.4 Subsystem Data Processing

The way these data structures are updated is through the MatchFrame event. That way, when any unit appears on vision, the unit begins to be tracked by the information manager and is then updated based on the new information we learn about it. To prevent a unit from being counted multiple times, BWAPI provides a unique ID for all units when they are created to be able to identify a specific unit.

5 Scouting Manager

In order to acquire information about the opponent, a player must command units to travel around the map to discover information covered by the fog of war. A unit traveling around the map causes the fog of war to fade away around the unit, revealing crucial information about the opponent that a player would not be able to see otherwise. By sending out units to scout, players can find out information about where opponent base(s) are located, the types of units they are creating, and what strategy the opponent is prioritizing. ProtoBot has multiple scouting scripts to allow units of different types to scout out information when commanded, and for different behaviors to be attempted based on the state of the script. To make the traversal of units across the map easier, the BWEM library is used to analyze the map to determine terrain navigation and provides some basic information on possible opponent base locations. This allows ProtoBot to know where possible starting locations are (possible starting locations are also shown to human players beforehand).

5.1 Layer Software Dependencies

BWAPI
BWEM
Misc. C++ Libraries

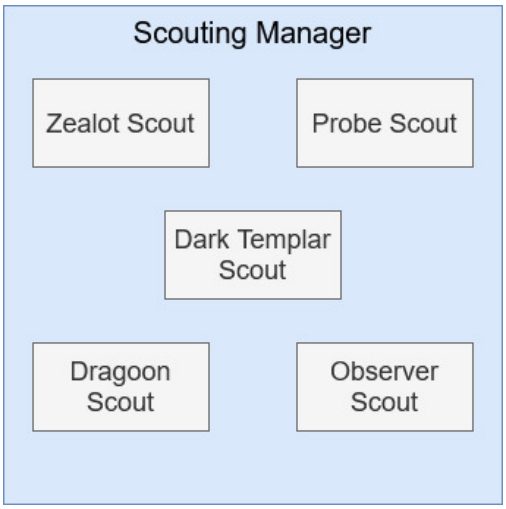

Figure 4: Scouting Manager subsystem diagram

5.2 Subsystem: Probe Scout

The Probe Scout subsystem defines the behavior of a probe when assigned to scouting. It is implemented as a state machine that enables a probe to switch between multiple behaviors including returning resources if carrying any, searching for an opponent base, gas stealing, harassment, and orbiting. The primary goal of a probe scout is to identify the opponent's main base location and gather early-game information while disrupting the opponent when possible.

5.2.1 Subsystem Software Dependencies

BWAPI
BWEM
A* Pathfinding Module
Misc. C++ Libraries

5.2.2 Subsystem Programming Languages

C++

5.2.3 Subsystem Data Structures

The `startTargets` vector contains candidate starting base positions to search. The `orbitWaypoints` vector stores positions around the enemy base used for circling behavior. The `plannedPath` vector contains the positions generated by the A* pathfinding system.

5.2.4 Subsystem Data Processing

The probe selected to scout is assigned by the Scouting Manager and performs the following instructions to locate the opponents base.

- **Return:** Returns gathered resources if carrying minerals or gas.
- **Search:** Visits possible starting locations.

Once a base is identified, the probe transitions between multiple states including:

- **Gas Steal:** Attempts to construct an Assimilator on the enemy geyser when approved by the strategy system.
- **Harass:** Targets enemy workers near the main base.
- **Orbit:** Circles the enemy base using predefined waypoints to maintain vision while avoiding threats.

Movement is handled using a hybrid of BWEM pathing and A* pathfinding. Threat detection is used to avoid enemy units and prolong a probe's lifetime.

5.3 Subsystem: Zealot Scout

The Zealot Scout subsystem defines the behavior of combat units (Zealots and Dragoons) used for mid-game scouting and map control. Unlike probe scouting, this subsystem focuses on positioning, map presence, and information gathering without entering high-risk areas.

5.3.1 Subsystem Software Dependencies

BWAPI
BWEM
Micro Utility Module
Misc. C++ Libraries

5.3.2 Subsystem Programming Languages

C++

5.3.3 Subsystem Data Structures

`enemyMainTile`, `enemyMainPos`, `enemyNaturalTile`, and `enemyNaturalPos` store known enemy locations. Additional internal state variables track patrol points, edge positions, and retreat states.

5.3.4 Subsystem Data Processing

Zealot and Dragoon scouts operate using a state-driven system that prioritizes map control and safe positioning. Their behaviors include:

- Moving to the enemy natural expansion and holding vision outside the base.
- Avoiding the enemy's main region to reduce the risk of being attacked.
- Repositioning when threats are detected using flee/kiting logic.
- Holding strategic vision points near expansion locations.
- Executing proxy patrol behavior by checking chokepoints and nearby regions for hidden structures.

The subsystem uses BWEM for area and path analysis and micro utility functions for threat avoidance and repositioning.

5.4 Subsystem: Observer Scout

The Observer Scout subsystem defines the behavior of observer units used for invisible scouting, map vision, and opponent detection support. Unlike ground-based scouts, observers are used to safely monitor high-value enemy locations over long periods of time. This subsystem uses a state-based controller combined with threat-aware movement to maintain vision while avoiding enemy detection and anti-air danger.

5.4.1 Subsystem Software Dependencies

BWAPI
BWEM
PurpleWave ThreatGrid Module
Misc. C++ Libraries

5.4.2 Subsystem Programming Languages

C++

5.4.3 Subsystem Data Structures

The subsystem stores `enemyMainTile` and `enemyMainPos` to track the enemy main base. The `posts` vector stores observer post locations, including the enemy main and nearby expansion locations. Additional fields store observer slot assignments, roaming checkpoints, current targets, and home positions for expansion-patrol behavior.

5.4.4 Subsystem Data Processing

Observers operate using a state-driven system with behaviors such as `Idle`, `MoveToPost`, `Hold`, and `AvoidDetection`. Their behaviors include:

- **Post Assignment:** Observers are assigned to predefined posts near the enemy main base and nearby expansions.
- **Hold Position:** Maintain vision at assigned posts while minimizing unnecessary movement.
- **Roaming Patrol:** A special observer slot patrols expansion locations not currently covered by other observers or friendly bases.

- **Detection Avoidance:** Move away from enemy detector units when entering or nearing their detection radius.
- **Threat Avoidance:** Query the ThreatGrid system to determine if positions are unsafe due to detector coverage or air threats.
- **Detour Movement:** If a direct path is unsafe, compute alternate movement paths toward safer intermediate positions.
- **Map Coverage:** Continuously gather information by maintaining vision over enemy bases and potential expansion locations.

The BWEM is used to identify reachable base locations and compute observer post positions. The ThreatGrid module maintains detection and threat maps based on enemy units, allowing observers to avoid dangerous areas while maintaining scouting effectiveness.

5.5 Subsystem: Dark Templar Scout

The Dark Templar Scout subsystem defines the behavior of Dark Templar units used for stealth scouting and targeted harassment. These units leverage invisibility to gather information and disrupt enemy operations without being detected.

5.5.1 Subsystem Software Dependencies

BWAPI
BWEM
Combat Targeting Utilities
Misc. C++ Libraries

5.5.2 Subsystem Programming Languages

C++

5.5.3 Subsystem Data Structures

Internal state variables track target positions, the enemy's main location, and current attack targets. Unit pairing logic may be used to coordinate multiple Dark Templar.

5.5.4 Subsystem Data Processing

Dark Templar units operate using a staged behavior system:

- **Approach:** Move toward the enemy natural expansion before entering the main base.
- **Infiltration:** Enter the enemy main base while remaining undetected.
- **Attack:** Prioritize high-value targets such as workers and detectors.
- **Persistence:** Maintain focus on a target during attack-move operations to maximize damage output.

If detection units are encountered, Dark Templar may prioritize eliminating detectors or repositioning. The subsystem emphasizes stealth, target prioritization, and sustained pressure inside enemy territory.

6 Economy Manager

In StarCraft, to place buildings, train troops, and consequently expand your influence on the map, the player must pay for these units using minerals and gas [Figure 14]. These two resources are harvested using worker units (specifically called "Probes" for Protoss) that travel to mineral deposits or vespene geysers with assimilators (Unique Protoss unit that is able to farm gas), collect the resource, deposit it at their assigned nexus, and repeat the process. The Economy Manager is the module that manages this process. It is responsible for the harvesting of resources and the management of probes. Additional functions of the Economy Manager include transferring probes to the Build Manager, Scouting Manager, or to another nexus when a new one is created or destroyed.

6.1 Layer Software Dependencies

BWAPI
Misc. C++ Libraries

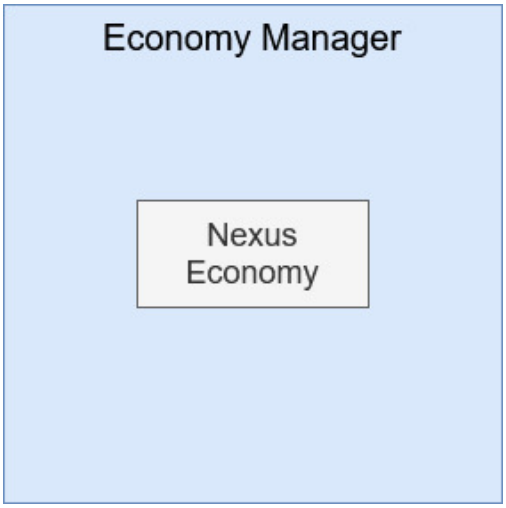

Figure 5: Economy Manager subsystem diagram

6.2 Subsystem: Nexus Economy

The Nexus Economy subsystem manages workers assigned to its resource depot (Protoss Nexus). This subsystem uses a resource-farming algorithm to determine the best resource for a worker to gather. Additionally, The subsystem is also responsible for the management of workers if one is needed to scout or for the construction of a building. A nexus economy also handles the transfer of workers to another nexus economy when a mineral is depleted or the nexus economy is destroyed, ensuring that workers are always performing productive actions.

6.2.1 Subsystem Software Dependencies

BWAPI

6.2.2 Subsystem Programming Languages

C++

6.2.3 Subsystem Data Structures

Within the Nexus Economy class, there are defined lists for the attributes of a base location. When a Nexus Economy is created at a base, the Economy Manager populates multiple lists with the mineral

deposits and vespene geysers located at the base as well. These lists determine the maximum number of workers that can be created for a given nexus economy and allow the strategy manager to produce the units required for that nexus economy.

6.2.4 Subsystem Data Processing

When a resource depot is completed, any nearby probes that are created in the future will be assigned to it. Assigned workers will then be commanded to farm a resource determined by the resource-farming algorithm, causing these workers to travel to a resource, farm for an amount of time set by StarCraft, return to their assigned nexus, and repeat the process. The algorithm used for the farming of minerals commands workers to wait for a mineral to be "open", instead of traveling to another open mineral immediately. When an assimilator is built on a vespene geyser at a base, a similar process is used to fetch vespene gas, where a maximum of 3 workers are assigned to farm gas.

7 Building Manager

The Build Manager handles the creation of units and placement of buildings based on the requests determined by the Strategy manager. When requests are made, the Build Manager acts as the second phase for the creation of units, confirming that a unit is actually created and allowing builders to be assigned in order to construct a building request.

7.1 Layer Software Dependencies

BWAPI
BWEB
C++ Libraries

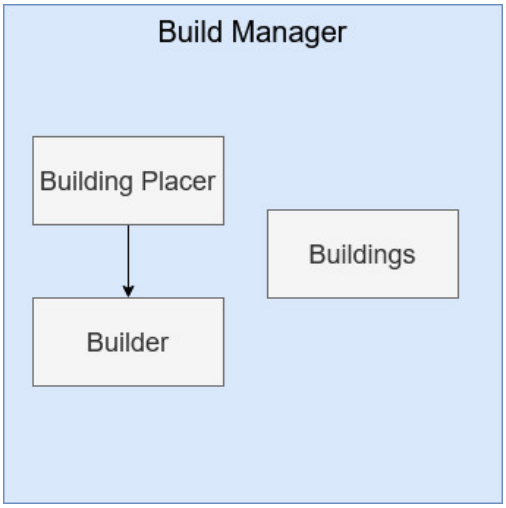

Figure 6: Building Manager subsystem diagram

7.2 Subsystem: Building Placer

The Building Placer subsystem determines the optimal location for a building to be constructed. This subsystem finds the closest location to construct a building based on how far a building location is from ProtoBot's main base and how far a location is away from an opponent's base. When BWEB is first initialized, an algorithm utilizing BWEM's map data is used to generate blocks of different sizes for the placement of buildings. 11 unique different blocks can be generated by the BWEB and are comprised of a combination of 4 x 3, 3 x 2, and 2 x 2 tile placements.

7.2.1 Subsystem Software Dependencies

BWEB

7.2.2 Subsystem Programming Languages

C++

7.2.3 Subsystem Data Structures

Valid block tile positions for the construction of buildings

7.2.4 Subsystem Data Processing

When determining the optimal placement of a building, the building placer requires the type of building requested to be constructed to find an optimal placement. Based on the type of unit passed, this allows

the building placer to then determine open tile positions in BWEB defined blocks to construct a building. In order for a placement to not override the construction of another unit, when a valid location to construct a building is found, the placer will then reserve the tile positions until a builder dies or a building is destroyed. If a valid placement cannot be found, the placer will then return an invalid position.

7.3 Subsystem: Builder

The Builder class is used to command a worker to be able to take optimal paths to be able to construct the buildings at specified locations from the Building Placer. Workers are the only units that are able to construct buildings, so when a building has been requested to be constructed, the Build manager requests that a worker be assigned temporarily as a builder to be able to perform the behavior for constructing a building.

7.3.1 Subsystem Programming Languages

C++

7.3.2 Subsystem Data Processing

When a Builder is created, the A-Star class is utilized to generate a path between a worker's current position and the position of where the building needs to be constructed. This allows a builder to iterate through each position in the generated path until it reaches the end position, taking a more optimal path than the one StarCraft would have generated. In the event that A-Star cannot generate a path, the default StarCraft move command is used to move a worker to the desired position to construct the building. When the worker reaches the position to construct a building, the worker "places" the building down and returns to its nexus.

7.4 Subsystem: Buildings

The Build Manager keeps track of all the ProtoBot buildings that are completed and are not destroyed in a vector called Buildings. This vector allows ProtoBot to determine how many buildings are in the process of being completed and at what time.

7.4.1 Subsystem Programming Languages

C++

8 Combat Manager

The Combat Manager is the layer which handles the combat between ProtoBot's troops and the enemy's troops. This combat is done through behaviors executed through a finite state machine that all squads have. Periodically, the Protobot Commander will receive actions from the strategy manager and call on the Combat Manager's commands. These commands, e.g. defend, attack, reinforce, etc., dictate what state squads enter or exit. Additionally, every state has a function called Update() that runs every frame, commanding units according to the current state of the squad.

8.1 Layer Software Dependencies

BWAPI

Figure 7: Combat Manager subsystem diagram

8.2 Subsystem: Squads

Squads are ProtoBot's main way of grouping combat units together. Each squad contains a finite state machine with 4 states: IDLE, DEFEND, REINFORCE, and ATTACK. Through these states, squads can defend key positions, reinforce nearby squads if attacked, and attack enemies with full force after certain conditions are met.

8.2.1 Subsystem Programming Languages

C++

8.2.2 Subsystem Data Structures

Every squad uses a state machine where each state contains an Enter(), Update(), and Exit() function. Enter() is ran when a squad first enters the state. Update() occurs every frame of the game. Exit() occurs when a squad is changing states.

8.2.3 Subsystem Data Processing

Combat Manager contains commands for ProtoBot Commander to call on when it receives actions from Strategy Manager. The manager will execute one of three commands: attack(...), defend(...), and reinforce(...). In order to avoid actions from Strategy Manager interrupting necessary state behavior, some commands only change the states of squads that are in specified states already (e.g., defend(...) only changes the states of squads in IDLE or ATTACK states).

The DEFEND state occurs often in the early game as it is meant for squads to move to defensive positions, e.g., choke points. The REINFORCE state occurs when a nearby squad is under attack. While in the REINFORCE state, squads will help the nearby squad being attacked without moving too far from their initial defensive positions. Finally, the ATTACK state triggers after a threshold is met for ProtoBot's unit count and supply count. This ATTACK state has units move towards enemy locations sent by Strategy Manager. In every state, squad's utilize kiting maneuvers to attack enemies while also moving away, prolonging their survival.

9 Strategy Manager

The strategy manager is responsible for deciding macro based actions to manipulation squads to defend ProtoBots buildings and for deciding when to attack the opponent. Every MatchFrame event received the strategy manager will populate a vector of actions for each module to perform giving the necessary information to guide ProtoBot's other modules to perform a specific action.

9.1 Layer Software Dependencies

BWAPI

Figure 8: Strategy Manager subsystem diagram

9.1.1 Subsystem Programming Languages

C++

9.1.2 Subsystem Data Structures

An action is a defined type that the Commander and Strategy Manager are able to understand to command ProtoBots module's what to do. There are 5 defined actions that ProtoBot can send:

- `ACTION_SCOUT`
- `ACTION_ATTACK`
- `ACTION_DEFEND`
- `ACTION_REINFORCE`
- `ACTION_NONE`

The real processing of the action happens in the commander where it figures out the action type and any other sub information that was passed to be able to perform the action.

9.1.3 Subsystem Data Processing

When a MatchFrame event is received from BWAPI, we first check if we are still performing a build order. If we are still performing a build order, the function returns and does not command any module to perform a specific task unless we can attack or defend with a squad. Once we complete a build order,

we then issue build commands for a majority of the frames in order to begin growing our army. Our bots' primary focus is on the creation of gateways, pylons, and nexuses. ProtoBot can create all units, but cannot utilize all the units that are created from them at this current iteration. When we want to construct a building, we first check if we have the necessary tech to construct a building. StarCraft has a mechanic called a "Tech Tree" [Figure 13], which requires the creation of a specific unit before a unit further down the tech tree is able to be created. The strategy manager keeps track of the count of a unit to make sure to follow the tech tree. If we have the necessary tech we construct buildings based on these rules, if we have a saturated Nexus Economy this allows us to support the constant production of roughly 4 gateways or 2 gateways and a stargate or robotics facility, if enough time has passed or we have a surplus of minerals we should place a nexus at an expansion to increase mineral and gas production, and if we are about to be supply capped create pylons to increase supply.

9.2 Subsystem: Spender Manager

The Spender Manager is the subsystem which keeps track of planned mineral and gas ProtoBot has reserved to spend and decides what units are able to be made with the remaining resources. The spender manager checks a list of resource requests given from the Commander and takes into account the minerals and gas to be able to create a unit and the priority of what units should be made over others. If there is enough available resources for the unit to created, the spender manager will reserve those minerals for the unit and move onto the next request. Resources requests are treated as first come first server and are placed in a specific order by the Strategy Modules as for what units take priority over others.

9.3 Subsystem Data Processing: Unit, Upgrade, and Building Production

In order to control address the balancing unit production and the construction of buildings. ProtoBot utilizes certain prioritizes to consider what units should be made every MatchFrame event received. ProtoBot consists of two types of production goals for units and upgrades.

Eight unit production goals are defined:

- `SATURATE_WORKERS`
- `EARLY_ZEALOTS`
- `DARK_TEMPLAR_ATTEMPT`
- `INFINITE_DRAGOONS`
- `OBSERVER_SCOUTS`
- `SOMETHING_WENT_WRONG_GO_INFINITE_ZEALOTS`
- `INVISIBLE_UNIT_DETECTED_SQUADS_NEED_OBSERVERS`
- `FLYING_UNIT_DETECTED_NEED_CANNONS`

Five upgrade production goals are defined:

- `RESEARCH_SINGULARITY_CHARGE`
- `RESEARCH_GROUND_WEAPONS`
- `RESEARCH_GROUND_ARMOR`

- RESEARCH_PLASMA_SHIELDS
- SOMETHING_WENT_WRONG_RESEARCH_LEG_ENHANCEMENTS

Each of the defined goals act as a way to control unit production from overproducing units of a certain type and works in tandem with building production to be able to project our income forward using the following rough approximation.

```
const double workerIncomePerFrameMinerals = 0.044;
const double workerIncomePerFrameGas = 0.069;
double ourIncomePerFrameMinerals = 0.0;
double outIncomePerFrameGas = 0.0;

activeMiners() is a check on the number of workers with the orders:
BWAPI::Orders::MoveToMinerals
BWAPI::Orders::WaitForMinerals
BWAPI::Orders::MiningMinerals
BWAPI::Orders::ReturnMinerals

activeDrillers() is a check on the number of workers with the orders:
BWAPI::Orders::MoveToGas
BWAPI::Orders::WaitForGas
BWAPI::Orders::HarvestGas
BWAPI::Orders::ReturnGas

//Estimate income
ourIncomePerFrameMinerals = workerIncomePerFrameMinerals * activeMiners();
outIncomePerFrameGas = workerIncomePerFrameGas * activeDrillers();
```

When a building wants to be created we will define the frame we want a building to start by and use the resources availible to not affect the start time of building to be placed in the future. While not the most ideal solution compared to other bots, production is one of the hardest features to get a human like balance to address the different types of things that can happen in a *StarCraft: Brood War* game.

10 Appendix

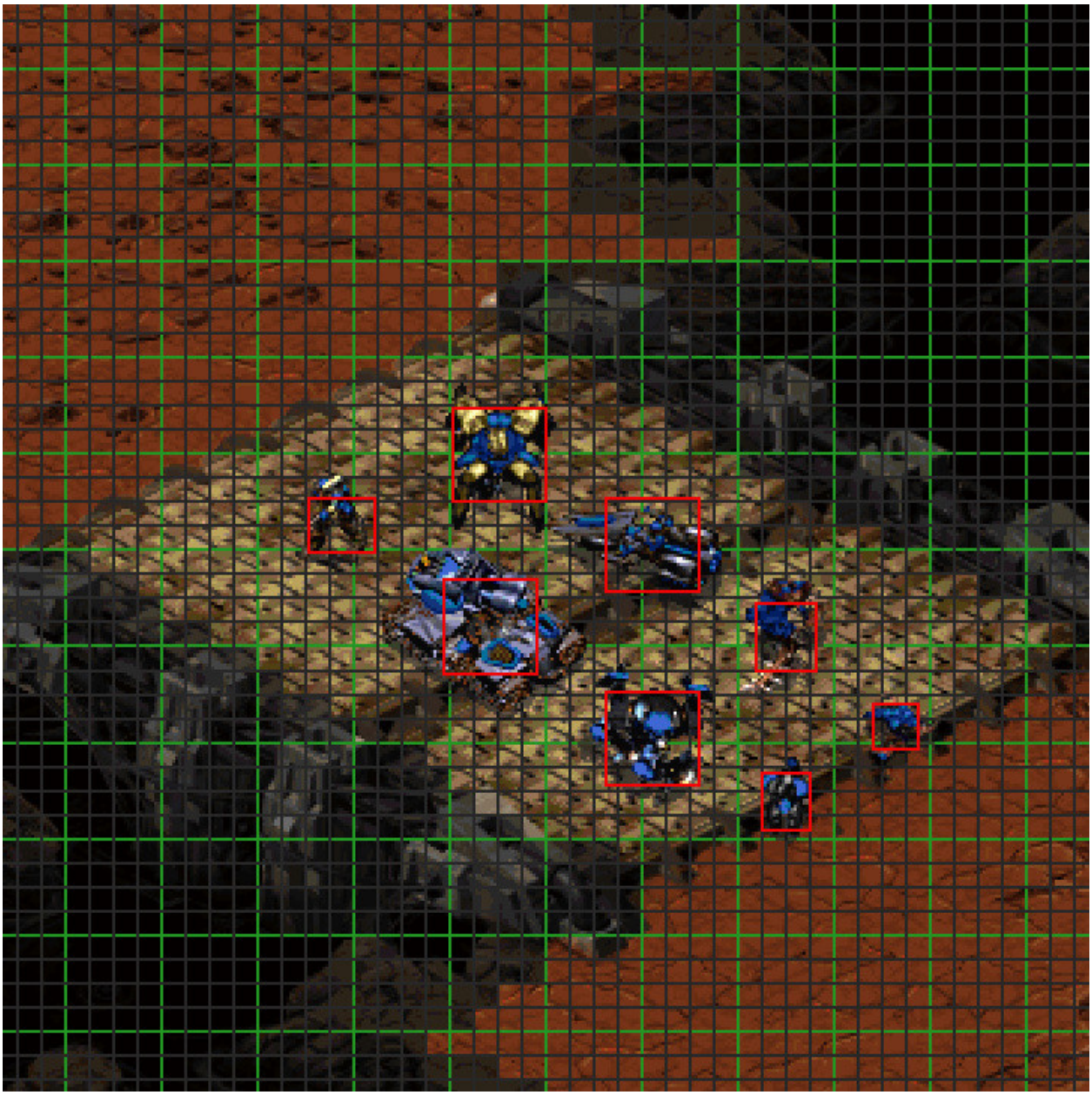

Figure 9: Example of tiles in StarCraft

Figure 10: Example of Fog of War in StarCraft

Figure 11: Example of sight range in StarCraft

Figure 12: Protoss unit tech tree

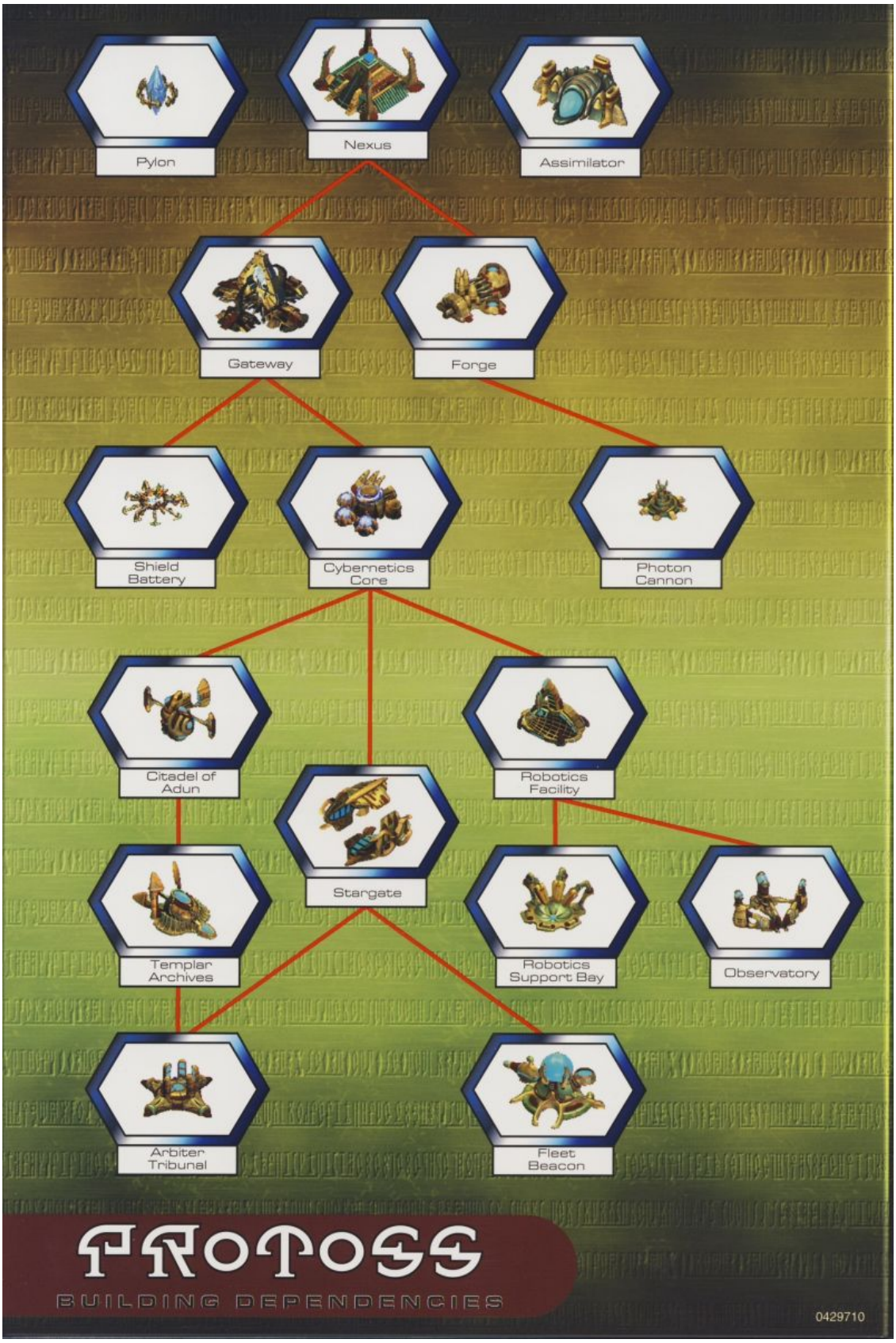

Figure 13: Protoss building tech tree

Figure 14: StarCraft resources (minerals and vespene gas)
Minerals on far right side and Vespene Geyser at the top.

Figure 15: Example of buildings being "remembered" by StarCraft map